Tail Latency

why the slowest 1% dominates at scale, and how to fight it

Watch the 4-min video

In one line: a request that waits for many parallel calls is as slow as the slowest one, so with 100 calls that are each slow 1% of the time, about 63% of requests are slow; fight the tail with hedged and tied requests, partial results at a deadline, less variance at the source, and measure p99/p99.9 at the user-facing level with merged histograms.

1 A fine p99 that isn't fine

Your search service has a p99 of 100 ms: only one request in a hundred is slow. That sounds fine. But each user search fans out to 100 shards. Now most user searches are slow.

2 Fan-out turns the tail into the common case

If a single call is slow 1% of the time and a request must wait for 100 of them, the chance that **all** are fast is $0.99^{100} \approx$ **37%**. So about **63%** of user requests hit at least one slow call. The more services and shards a request touches, the more the tail matters.

$$P(\text{request slow}) = 1 - (1 - p)^n \quad p = 1\%,\ n = 100 \rightarrow 63\%$$

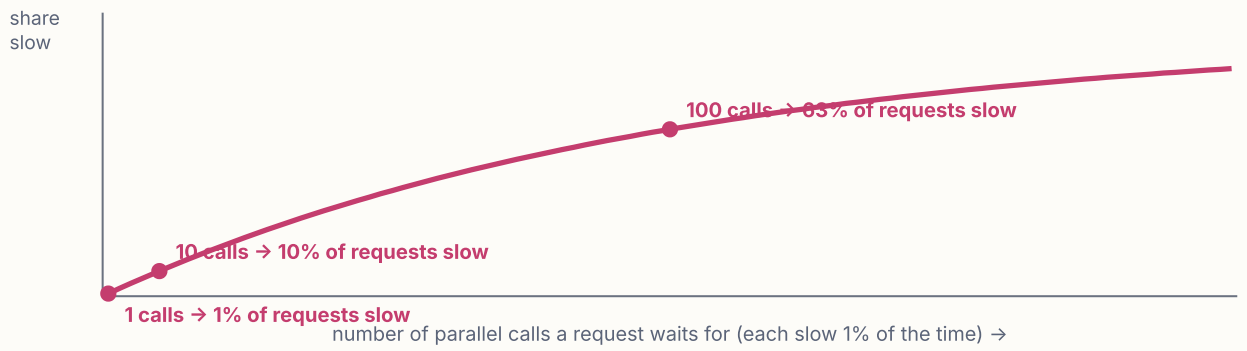


Figure 1. The share of requests that hit at least one slow call grows quickly with fan-out.

3 Where slow calls come from

Garbage-collection pauses. Noisy neighbours on shared hardware. Queueing behind a burst (lesson 45). Background work such as compaction or log rotation. Cache misses and cold starts. Network retransmissions. Most are random and short, which is exactly why they're hard to remove: at scale, something somewhere is always having a bad moment.

4 Hedged and tied requests

Hedged requests: send the request to one replica; if it hasn't answered by about the **p95** latency, send a second copy to another replica and use whichever answers first (cancel the other). Only about 5% of requests are duplicated, but the tail shrinks dramatically. Do this only for **idempotent reads**. Google described these techniques in *The Tail at Scale* (Dean and Barroso, 2013).

Tied requests go further: send to two replicas at once, each told about the other; when one starts processing, it cancels its twin, so duplicates rarely do real work.

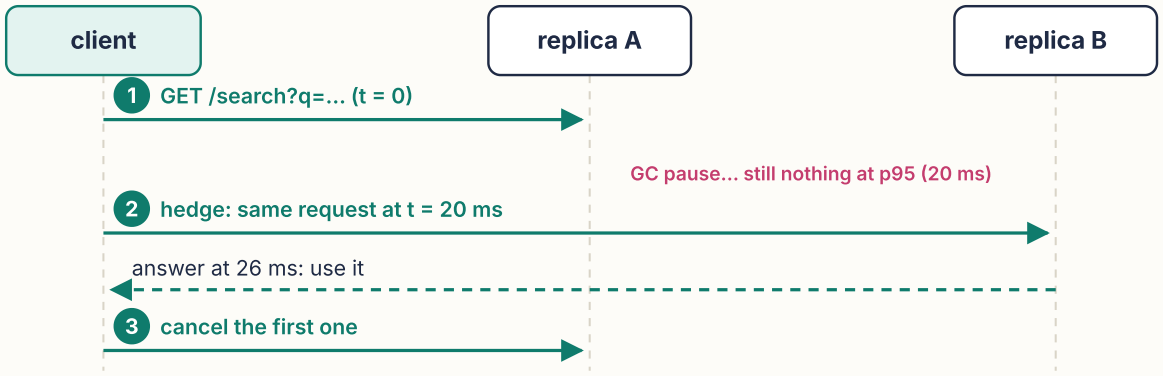


Figure 2. A hedged request: the first replica stalls, the hedge to a second replica answers, and the first is cancelled.

5 More tools

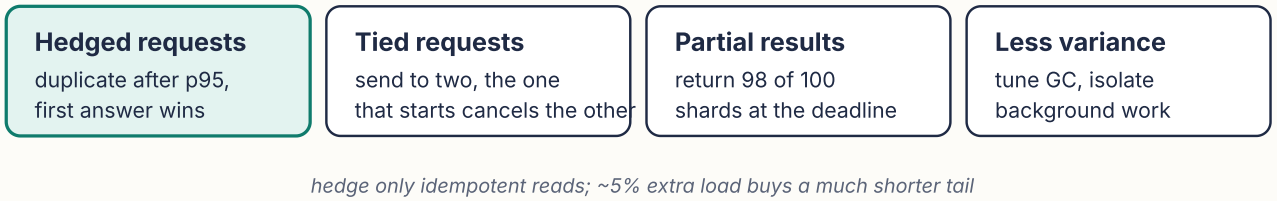


Figure 3. Four ways to cut the tail.

Tight timeouts with retries (lesson 9), returning **partial results** when a few shards are late, and reducing variability at the source: tuning garbage collection, isolating background work on separate machines or time windows, and keeping queues short.

WORKED EXAMPLE: SEARCH ACROSS 100 SHARDS

A query fans out to 100 shards, and the slowest one decides the response time. With hedging after the p95 latency, a slow shard is quietly replaced by its replica. With a deadline, the service returns results from 98 shards rather than wait for the last two (and says so). The user-facing p99 drops from seconds to a few hundred milliseconds.

6 Measure the tail properly

- Track **p99 and p99.9**, not averages.
- Measure at the **user-facing** level, not only per service.
- **Never average percentiles** across servers: the average of ten servers' p99s is not the fleet's p99. Merge histograms (Prometheus histograms, HDRHistogram) instead.

COMMON MISTAKES

- Judging a fan-out system by per-backend p99.
- Hedging writes, causing duplicates.
- Hedging immediately (not after a delay), doubling load for nothing.
- Averaging percentiles on dashboards.

7 Key terms

Tail latency: the slowest requests: p99, p99.9

Hedged request: a delayed duplicate to another replica

Partial results: answer without the slowest shards at a deadline

Fan-out: one request calling many backends in parallel

Tied request: two copies that cancel each other on start

Histogram merge: the right way to combine percentiles

8 Interview questions

Q1. Why does fan-out amplify tail latency? BASIC

A request waits for its slowest call, so with 100 calls each slow 1% of the time, about 63% of requests hit at least one slow call.

Q2. What are hedged requests? BASIC

After a short delay, like the p95 latency, send a duplicate to another replica and use the first response, cutting the tail for a small extra load.

Q3. Your API calls 5 services in parallel, each with p99 = 200 ms. What's the user-facing p99 roughly? SCENARIO

The chance all five are under their p99 is 0.99^5 ≈ 95%, so about 5% of requests exceed 200 ms: the user-facing p99 is well above 200 ms, closer to the services' p99.8.

Q4. How would you cut the p99 of a search over 50 shards? SCENARIO

Hedge slow shard calls after the p95, set a deadline and return partial results, reduce variance on the shards (GC, background compaction), and keep shard queues short.

Q5. Why can't you average p99 across servers? TRICKY

Percentiles don't add or average; one server with terrible latency can dominate the real fleet p99 while the average looks fine. Merge the histograms and compute the percentile from the combined data.

Q6. When is hedging a bad idea? TRICKY

For non-idempotent operations, when the backend is overloaded (duplicates add load at the worst time; use a budget), or when slowness is caused by the request itself, so the hedge is just as slow.

Next: SD-J-014 · Performance Optimization Strategy backendroadmap.com